

Al terminar todo: pedir mejoras del seo, rendimiento, estéticas generales que me proponga claude con las skills, también mejora ux/ui, accesibilidad, y por último de tood cuando ya vea todo seguridad, en general y de formularios

Pendiente

Cruces en rojo antes de tocar, cambiar la sección de info para poner bien las palabras largas, preguntar por mejoras estéticas, seguridad y seo, necesito un panel de administración para las noticias/novedades
info fecha de inicio, y diciendo que todavía tiene tiempo para inscribirte
formulario en sobre nosotros
texto de nuevo en nuevas carreras

agregar sección de preguntas frecuentes y de identidad argentina
angulo de inclinación bnadera argentina, efecto hover flameando
falta hero section en index.html?
convenios correlatividades otras universidades
carga dinámica de carreras? mejor o peor
en la sección novedades incluir los cursos!

PASO 6 (OPCIONAL): Configurar Context7 MCP (10 min) - GRATIS

 **COSTO: \$0** - Tier gratis con 500 requests/mes

 **IMPORTANTE:** Solo hazlo en la Semana 2 cuando vayas a migrar a React/Next.js (Fase 4)

1. Crear cuenta GRATIS (opcional pero recomendado)

Ve a: <https://context7.com/dashboard>

Regístrate (gratis) para obtener API key

2. Configurar Context7 (funciona SIN API key también)

claude mcp add context7 -- npx -y @upstash/context7-mcp --api-key TU_API_KEY

3. Editar configuración (añadir a los MCPs existentes)

nano ~/.config/claude/mcp.json

Añade context7:

```
{
  "mcpServers": {
    "github": { /* ... */ },
    "brave-search": { /* ... */ },
    "context7": {
```

```

    "command": "npx",
    "args": ["-y", "@upstash/context7-mcp", "--api-key", "TU_API_KEY_CONTEXT7"]
  }
}
}

```

¿Para qué sirve? Acceso a documentación oficial actualizada de Next.js, React, Tailwind, etc.

Límites del tier gratis:

- 500 requests/mes GRATIS
- Úsalo solo en Fase 4 (React/Next.js)
- Di explícitamente "use context7" en tu prompt
- Estimado: ~20-30 requests en toda la Fase 4
- Si necesitas más: Plan Pro \$7/mes

PASO 7: Verificar Skills Oficiales (2 min) - OPCIONAL

skills.sh de versalles, usar

inicio token: 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

•

Context7 MCP - Úsalo explícitamente en Fase 4 (React/Next.js):

claude "usa context7 para implementar App Router de Next.js 14"
 # Claude consultará docs oficiales de Next.js 14

•

- **Skills** - Se usan automáticamente cuando creas archivos:
 - Tu skill **cau-brand** se aplicará en TODOS los componentes React
 - Skills oficiales (docx, pdf, etc.) se usan cuando generas documentos

•

Comandos a ejecutar:

Pendiente: que los enlaces de redes sociales y whatsapp lleven realmente a los números

Las activo automáticamente según lo que me pidas. Por ejemplo:

- Si me pedís "mejorá el diseño de la página" → uso frontend-design o ui-ux-pro-max
- Si me pedís "revisá el SEO" → uso seo-audit
- Si me pedís "quiero agregar una nueva sección" → primero uso brainstorming para entender qué necesitás, y luego implemento
- Si me pedís "revisá la accesibilidad" → uso web-design-guidelines

Simplemente decime qué querés hacer y yo elijo la herramienta adecuada.

Fix 2: Carousel

Pendiente: darle mejor estética a las tarjetas del carrousel/poner imagenes en cada una

Fix 3: Modal flotante

claude "el modal en pc es algo corto, quizás vendría bien agrandarlo y botón cerrar rojo"

Fix 4: Ocultar contador

claude "ocultar completamente el contador de clicks del código"

Guardar cambios

git add .

git commit -m "feat: correcciones críticas UX (menú, carousel, modal, contador)"

git push

Limpiar contexto

claude reset



DÍA 2 - FASE 2: Mejoras UX/UI (1-2 horas)

Objetivos:

- ☒ Rediseñar estética del buscador
- ☒ Centrar "Lo más buscado" y botones
- ☒ Hover con imágenes en tarjetas
- ☒ Cambiar texto "Buscar carrera"
- ☒ Scroll preciso de botones a secciones

Comandos a ejecutar:

Iniciar sesión

claude init

Mejora 1: Buscador

claude "rediseña el buscador con estética premium usando colores #00c7b1 y #013729, con mejor tipografía y espaciado"

Mejora 2: Centrado

claude "centra perfectamente el título 'Lo más buscado' y todos los botones de categorías (Ver Todo, Licenciaturas, etc)"

Mejora 3: Pendiente:

claude "en tarjetas 'Más Buscadas': al hacer hover mostrar imagen relacionada con la carrera usando smooth transition"

Mejora 4: Texto buscador

claude "cambia el texto 'Buscar carrera' por algo más atractivo y profesional"

Mejora 5: Scroll preciso

claude "cuando se hace click en botones de categorías, deben hacer scroll suave y preciso a su sección correspondiente"

Guardar cambios

git add .

git commit -m "feat: mejoras UX/UI (buscador, centrado, hover, navegación)"

git push

claude reset

BONUS (Opcional - +30 min):

claude "añadir loading states animados en las búsquedas"

claude "añadir feedback visual (escala, brillo) en todos los botones al hacer click"

claude "transiciones suaves entre secciones con scroll-behavior smooth"

Pausa de 5 horas

DÍA 3 - FASE 3: Contenido Nuevo (2-3 horas)

Objetivos:

-  Sección Clases de Apoyo premium
-  Sección Cursos
-  Badge "¡NUEVO!" en Identidad Argentina

-  Nota Amigo Referido/Deportista

Comandos a ejecutar:

claude init

Sección 1: Clases de Apoyo

claude "crea sección 'Clases de Apoyo' con diseño premium, usar color #e69b05 para destacar, incluir tarjetas con materias y horarios"

Sección 2: Maestrías

claude "crea sección 'Maestrías' siguiendo el mismo estilo de las secciones existentes, con grid de tarjetas"

Sección 3: Cursos

claude "crea sección 'Cursos' con diseño consistente, incluir categorías y filtros básicos"

Badge NUEVO

claude "añadir badge animado '!NUEVO!' con color #dab03a y efecto pulse en la carrera Identidad Argentina"

Nota Amigo Referido

claude "crear sección destacada para beneficio 'Amigo Referido o Deportista Federado' con color #7c207b y diseño llamativo"

Guardar

git add .

git commit -m "feat: nuevas secciones (Clases Apoyo, Maestrías, Cursos, badges)"

git push

claude reset



BONUS (Opcional - +1 hora):

claude "añadir sección de Testimonios con carousel de estudiantes"

claude "crear FAQ expandible con accordion"

claude "añadir timeline visual del proceso de inscripción"

claude "counter animado de estudiantes matriculados"



Pausa hasta el día siguiente



DÍA 4-5 - FASE 4: Migración a React/Next.js (4-6 horas)

⚠ IMPORTANTE: Esta es la fase más pesada. Hazlo componente por componente.

Objetivos:

- ☒ Setup Next.js 14 + TypeScript + Tailwind
- ☒ Componentizar todo el HTML
- ☒ Implementar shadcn/ui
- ☒ Integrar Framer Motion
- ☒ Migrar estilos

Comandos a ejecutar:

claude init

PASO 1: Setup inicial (30 min)

claude "crear nuevo proyecto Next.js 14 con TypeScript y Tailwind CSS pre-configurado, usar App Router (no Pages Router)"

PASO 2: Configurar shadcn/ui (15 min)

claude "configurar shadcn/ui en el proyecto con los componentes: button, card, dialog, input"

PASO 3: Convertir Navbar (30 min)

claude "convertir el navbar HTML a componente React TypeScript en src/components/Header/Navbar.tsx"

claude "convertir menú mobile a componente MobileMenu.tsx con estado de apertura/cierre"

PASO 4: Convertir Hero (20 min)

claude "convertir sección hero a componente React en src/components/Hero/Hero.tsx"

PASO 5: Convertir Carousel (45 min)

claude "convertir carousel a componente React con Framer Motion en src/components/Carousel/Carousel.tsx, mantener funcionalidad de auto-scroll"

PASO 6: Convertir Buscador (30 min)

claude "convertir buscador a componente React en src/components/Search/SearchBar.tsx con estado y filtrado"

PASO 7: Convertir Tarjetas de Carreras (40 min)

claude "convertir tarjetas de carreras a componente CareerCard.tsx reutilizable con props"

PASO 8: Convertir Modal (25 min)

claude "convertir modal de carreras a componente React usando shadcn Dialog"

PASO 9: Convertir Secciones (1 hora)

claude "convertir sección 'Más Buscado' a src/components/Sections/MasBuscado.tsx"

claude "convertir sección 'Clases de Apoyo' a src/components/Sections/ClasesApoyo.tsx"

claude "convertir sección 'Maestrías' a src/components/Sections/Maestrias.tsx"

claude "convertir sección 'Cursos' a src/components/Sections/Cursos.tsx"

PASO 10: Crear Layout Principal (20 min)

claude "crear src/app/layout.tsx con metadata para SEO y estructura general"

PASO 11: Crear Página Principal (15 min)

claude "crear src/app/page.tsx que componga todos los componentes creados"

PASO 12: Migrar estilos y colores (30 min)

claude "migrar todos los estilos CSS a Tailwind, crear archivo de configuración con colores personalizados (#00c7b1, #013729, etc)"

Guardar

git add .

git commit -m "feat: migración completa a Next.js + React + TypeScript"

git push

claude reset



TIP: Si algún componente falla, usa:

claude "debug componente X - mostrar errores y solucionarlos"



Pausa de 5 horas



DÍA 6-7 - FASE 5: Backend + Features Avanzadas (4-6 horas)

Objetivos:

- ☒ Setup Express + MongoDB
- ☒ API REST de carreras
- ☒ Sistema de búsqueda
- ☒ Analytics de clicks
- ☒ Autenticación JWT
- ☒ Validación con Zod

Comandos a ejecutar:

claude init

PASO 1: Setup Backend (30 min)

claude "crear carpeta /server con Express.js, configurar MongoDB con Mongoose, estructura MVC (models, routes, controllers)"

PASO 2: Modelo de Carreras (20 min)

claude "crear modelo Mongoose para Carreras con campos: nombre, descripción, duración, modalidad, categoría, clicks"

PASO 3: API REST (45 min)

claude "crear API REST completa para carreras: GET /api/careers (listar), GET /api/careers/:id (detalle), POST /api/careers (crear), PUT /api/careers/:id (actualizar), DELETE /api/careers/:id (eliminar)"

PASO 4: Búsqueda Full-Text (30 min)

claude "implementar búsqueda full-text en MongoDB para buscar carreras por nombre, categoría, modalidad"

PASO 5: Analytics de Clicks (25 min)

claude "crear endpoint POST /api/careers/:id/click que incremente contador de clicks y registre timestamp"

PASO 6: Autenticación JWT (1 hora)

claude "implementar sistema de autenticación con JWT: registro, login, middleware de protección, hash de passwords con bcrypt"

PASO 7: Validación con Zod (30 min)

claude "añadir validación con Zod en todos los endpoints: validar estructura de datos antes de procesarlos"

PASO 8: Conectar Frontend con Backend (45 min)

claude "conectar componentes React con el backend usando fetch o axios, manejar estados de carga y errores"

PASO 9: Variables de Entorno (15 min)

claude "configurar variables de entorno (.env) para MongoDB URI, JWT secret, puerto del servidor"

Guardar

git add .

git commit -m "feat: backend completo con API REST, MongoDB, JWT, validación"

git push

claude reset

BONUS Backend (Opcional - +2 horas):

claude "implementar rate limiting con express-rate-limit para evitar spam"

claude "configurar CORS correctamente para producción"

claude "añadir Helmet.js para seguridad HTTP"

claude "implementar logging con Morgan"

claude "comprimir respuestas con compression middleware"



PARTE 3: FINALIZACIÓN Y DEPLOY

PASO FINAL 1: Testing (1 hora)

claude init

Test componentes

claude "crear tests unitarios básicos para componentes principales con Jest y React Testing Library"

Test API

claude "crear tests para endpoints del backend usando Supertest"

Test E2E

claude "crear test E2E del flujo completo de búsqueda e inscripción con Puppeteer"

PASO FINAL 2: Optimización (1 hora)

Performance

claude "implementar lazy loading de imágenes con next/image"

claude "optimizar bundle size con code splitting dinámico"

claude "configurar ISR (Incremental Static Regeneration) para páginas de carreras"

SEO

claude "generar sitemap.xml dinámico"

claude "implementar schema.org markup para carreras"

claude "configurar meta tags optimizados por página"

PASO FINAL 3: Documentación (30 min)

README

claude "genera README.md completo con: descripción del proyecto, instalación, uso, tecnologías, estructura de carpetas"

Documentación API

claude "documenta todas las APIs del backend en formato Markdown con ejemplos de requests/responses"

Guía de despliegue

claude "crea guía de deployment para Vercel (frontend) y Railway (backend)"

PASO FINAL 4: Deploy (30 min)

Frontend en Vercel

1. Ve a <https://vercel.com>

2. Import tu repo de GitHub

3. Vercel detectará Next.js automáticamente

4. Deploy

Backend en Railway

1. Ve a <https://railway.app>

2. New Project → Deploy from GitHub

3. Selecciona tu repo

4. Añade MongoDB como servicio

5. Configura variables de entorno

6. Deploy

Conectar Frontend con Backend

claude "actualizar las URLs del frontend para usar la URL de producción del backend de Railway"



RESUMEN EJECUTIVO

Configuración y Tiempo Total:

Configuración Inicial (Una sola vez):

- **Esencial (obligatorio):** 30 min (Pasos 1-4 + verificación)
- **Recomendado:** +20 min (Brave Search MCP + Skills)
- **Opcional:** +10 min (Context7 MCP - para Semana 2)

Tiempo Total por Fase:

- Configuración inicial: 30-60 min (una sola vez)
- Fase 1 (Correcciones): 30-60 min
- Fase 2 (UX/UI): 1-2 horas
- Fase 3 (Contenido): 2-3 horas
- Fase 4 (React/Next.js): 4-6 horas
- Fase 5 (Backend): 4-6 horas
- Finalización: 3 horas

TOTAL: ~16-22 horas de trabajo activo en 5-7 días

MCPs Configurados:

GitHub MCP (ESENCIAL)

- Commits automáticos
- Gestión de branches
- Control de versiones integrado

Brave Search MCP (RECOMENDADO)

- Búsquedas en internet cuando Claude no sepa algo
- Documentación actualizada
- Mejores prácticas 2025

Context7 MCP (OPCIONAL - Semana 2)

- Docs oficiales de Next.js, React, Tailwind
 - Información de versión específica
 - APIs actualizadas
-

Skills Disponibles:

Skills Oficiales (Ya instaladas, OPCIONALES para web):

- **docx** - Documentos Word (útil para: documentación del proyecto)
- **pdf** - Procesar PDFs (útil para: certificados de inscripción)
- **pptx** - Presentaciones (útil para: pitch deck del proyecto)
- **xlsx** - Hojas de cálculo (útil para: reportes de estudiantes)

 **NO son necesarias para desarrollo web HTML/CSS/JS/React**

Skill Personalizada (RECOMENDADA  para desarrollo web):

- **cau-brand** - Guías de marca de tu proyecto
 -  Colores consistentes (#00c7b1, #013729, etc.)
 -  Tipografía (Inter)
 -  Patrones de componentes React
 -  Principios de diseño
 -  **Esta SÍ es importante para tu proyecto web**
-

Prompts Totales Estimados:

- Fase 1: ~15 prompts
- Fase 2: ~20 prompts
- Fase 3: ~25 prompts
- Fase 4: ~60 prompts
- Fase 5: ~40 prompts
- Finalización: ~20 prompts

TOTAL: ~180 prompts

Con el plan Pro (\$20/mes) tienes ~40 prompts cada 5 horas. Distribuyendo bien en 3 sesiones/día = ~120 prompts/día

✓ **El plan Pro es SUFICIENTE si trabajas 5-7 días**

Orden Recomendado de Ejecución:

1. **DÍA 0:** Configuración inicial (40 min)
 2. **DÍA 1:** Fase 1 completa (1 hora)
 3. **DÍA 2:** Fase 2 completa (2 horas)
 4. **DÍA 3:** Fase 3 completa (3 horas)
 5. **DÍA 4-5:** Fase 4 completa (6 horas, 3h cada día)
 6. **DÍA 6-7:** Fase 5 completa (6 horas, 3h cada día)
 7. **DÍA 8:** Finalización y Deploy (3 horas)
-

Estrategia de Sesiones (Aprovecha el Reset de 5 horas):

SESIÓN 1 (9:00 AM - 11:00 AM):

- Trabaja ~40 prompts
- claude reset al terminar

PAUSA DE 5 HORAS 🕒 (límite se resetea automáticamente)

SESIÓN 2 (4:00 PM - 6:00 PM):

- Otros ~40 prompts
- claude reset al terminar

PAUSA DE 5 HORAS 🕒

SESIÓN 3 (11:00 PM - 12:00 AM) - OPCIONAL:

- ~20 prompts si quedó algo pendiente

TOTAL: ~100-120 prompts/día sin problemas 

Comandos de Referencia Rápida:

Al iniciar cada sesión

claude init

claude status

Al terminar cada sesión

git add .

git commit -m "descripción de cambios"

git push

claude reset

Si algo sale mal

claude logs

claude debug "descripción del problema"

Ver cuántos prompts llevas usado

claude usage

Ver historial

claude history

Checklist de Verificación (Marca conforme avanzas):

Configuración Básica (Obligatoria):

- ☐ Claude Code instalado y verificado
- ☐ Autenticado correctamente
- ☐ Proyecto inicializado (`claude init`)
- ☐ `.claudeignore` creado
- ☐ `claude.config.json` creado
- ☐ Git configurado y primer commit hecho
- ☐ Repositorio GitHub creado y conectado
- ☐ GitHub MCP configurado y funcionando
- ☐ Test inicial pasado

Configuración Recomendada (Opcional):

- ☐ Brave Search MCP configurado
- ☐ Context7 MCP configurado (para Semana 2)
- ☐ Skills oficiales verificadas (docx, pdf, pptx, xlsx)
- ☐ Skill personalizada **cau-brand** creada (para Semana 2)

Desarrollo:

- ☐ Fase 1 completada (Correcciones críticas)
- ☐ Fase 2 completada (UX/UI)
- ☐ Fase 3 completada (Contenido nuevo)
- ☐ Fase 4 completada (React/Next.js)
- ☐ Fase 5 completada (Backend)

Finalización:

- ☐ Tests creados
- ☐ Optimizaciones aplicadas
- ☐ Documentación generada
- ☐ Deployed en producción

¡ESTÁS LISTO PARA EMPEZAR!

Primer comando a ejecutar ahora mismo:

```
cd /ruta/a/tu/proyecto
```

```
claude init
```

```
claude "analiza mi index.html y dame un análisis de 5 puntos de lo que hay que mejorar"
```

¡Mucho éxito con tu proyecto! 🚀